

Are We All Using Agents the Same Way? An Empirical Study of Core and Peripheral Developers' Use of Coding Agents

Shamse Tasnim Cynthia
University of Saskatchewan
Saskatoon, SK, Canada
shamse.cynthia@usask.ca

Joy Krishan Das
University of Saskatchewan
Saskatoon, SK, Canada
joy.das@usask.ca

Banani Roy
University of Saskatchewan
Saskatoon, SK, Canada
banani.roy@usask.ca

Abstract

Autonomous AI agents are transforming software development and redefining how developers collaborate with AI. Prior research shows that the adoption and use of AI-powered tools differ between core and peripheral developers. However, it remains unclear how this dynamic unfolds in the emerging era of autonomous coding agents. In this paper, we present the first empirical study of 9,427 agentic PRs, examining how core and peripheral developers use, review, modify, and verify agent-generated contributions prior to acceptance. Through a mix of qualitative and quantitative analysis, we make four key contributions. First, a subset of peripheral developers use agents more often, delegating tasks evenly across bug fixing, feature addition, documentation, and testing. In contrast, core developers focus more on documentation and testing, yet their agentic PRs are frequently merged into the main/master branch. Second, core developers engage slightly more in review discussions than peripheral developers, and both groups focus on evolvability issues. Third, agentic PRs are less likely to be modified, but when they are, both groups commonly perform refactoring. Finally, peripheral developers are more likely to merge without running CI checks, whereas core developers more consistently require passing verification before acceptance. Our analysis offers a comprehensive view of how developer experience shapes integration offer insights for both peripheral and core developers on how to effectively collaborate with coding agents.

CCS Concepts

• **Software and its engineering** → **Collaboration in software development.**

Keywords

Coding Agents, Core Developers, Peripheral Developers, Pull Requests, Code Review, Code Modifications

ACM Reference Format:

Shamse Tasnim Cynthia, Joy Krishan Das, and Banani Roy. 2026. Are We All Using Agents the Same Way? An Empirical Study of Core and Peripheral Developers' Use of Coding Agents. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3793302.3793377>



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2474-9/26/04
<https://doi.org/10.1145/3793302.3793377>

1 Introduction

AI-powered tools such as copilots and coding agents have significantly advanced AI-assisted software development, drawing significant attention from both industry [74] and academia [12, 97]. While copilots primarily provide chat-based assistance that guides developers as they make changes within their own development environment, coding agents put forward greater autonomy. These agents operate within their ephemeral development environment, execute and adjust their actions based on feedback from compilers or linters to autonomously resolve issues and submit pull requests (PRs). This promises faster delivery cycles [11] and reduced cognitive load [80], leading to their growing adoption by developers across all experience levels [24, 90].

Despite increasing agent adoption, prior research has primarily examined chat-based assistance such as Copilot and ChatGPT. For instance, Barke et al. [9] showed that developers use Copilot either to speed up or to explore options when unsure. Similarly, a study with student programmers [71] found that despite challenges interpreting Copilot's suggestions, participants remained optimistic about its future use. These studies, in their early forms, highlight how developers feel about AI-assistance. While these studies offer useful insights, we still lack empirical evidence on how different types of developers interact with autonomous coding agents that behave more like fellow engineers [15].

This understanding is particularly important as most assessments on coding agents stem from benchmarks like SWE-Bench [41, 91, 93, 97]. Furthermore, some prior studies have shown that developers' perceptions and adoption patterns of AI-powered tools vary significantly with experience. For instance, Peng et al. [69] showed that peripheral developers completed tasks 55.8% faster with Copilot in a controlled study. Conversely, McKinsey [18] and Jellyfish [17] found that core developers benefited more substantially compared to junior developers (e.g., 7-10% more time to complete tasks), and Vaithilingam et al. [88] observed no significant experience-based differences. Collectively, these contrasting findings reveal that developer experience plays a critical role in shaping how AI-powered tools are adopted and utilized. However, we still lack empirical evidence on how this dynamic unfolds between core and peripheral developers when working with autonomous coding agents in real-world workflows.

To address this gap, we conduct an empirical study of 9,427 resolved agentic PRs on GitHub attributed to four widely used agents (Claude, Copilot, OpenAI Codex, and Cursor). Our aim is to understand how collaboration with agents differs between core and peripheral developers across the pull request (PR) lifecycle (i.e., from submission to acceptance). We proceed in four steps. First, we quantify how often developers delegate tasks to agents and for what

purposes. Second, we assess how intensely agent-generated code is reviewed and what issues are raised. Third, we measure how much and what types of developer modifications are applied to agent code. Last, we evaluate to what extent the merge verification check passes at acceptance. Our analysis offers a comprehensive view of how developer experience shapes delegation, review, modification, and verification, providing practical guidance for collaborating with agents to achieve reliable contributions across experience levels.

This paper makes four major contributions by answering four research questions as follows.

☞ **RQ₁. To what extent do core and peripheral developers differ in the frequency and purpose of agent use on GitHub?**

Although trust is inherently subjective, prior studies suggest that it can be inferred from an individual's usage of a system [7, 46] and the nature of tasks they are willing to delegate [77]. To investigate how these behaviors vary across core and peripheral developers, we analyze 9,427 closed agentic PRs. We find that, although usage rates are similar, a subset of peripheral developers uses agents disproportionately more often. Finally, peripheral developers delegate tasks evenly across *bug fixing*, *feature addition*, *documentation*, and *testing*), whereas core developers focus more on *documentation* and *testing*.

☞ **RQ₂. How do core and peripheral developers differ in the intensity of code review and the types of issues they raise on agentic PRs?**

Rigorous code review is known to improve software quality [62]. We therefore examine how the intensity of review comments and the types of issues raised on agentic PRs differ between core and peripheral developers. We find that core developers engage slightly more in review discussions than peripheral contributors. Both groups primarily address evolvability-related issues in agent-generated code. Within this category, peripheral developers tend to focus more on *code organization*, while core developers more frequently prioritize *alternative solution* approaches.

☞ **RQ₃. How do core and peripheral developers differ in the intensity and types of modifications applied to agent-generated code?**

After developer's review (RQ₂), agents may apply changes. However, it remains unclear whether developers still intervene to ensure that the agent's code adheres to project-specific standards. Therefore, we analyze how core and peripheral developers modify agentic PRs, measuring both the intensity and types of modifications made to agent's code. We find that, on average, 74.1% of agentic PRs are accepted without modification across both groups. When modifications do occur, both core and peripheral developers commonly *refactor agent-generated code*.

☞ **RQ₄. What differences exist in verification outcomes for agentic PRs accepted by core versus peripheral developers?**

Modern Code Review (MCR) [62, 85] is increasingly paired with automated verification checks through Continuous Integration (CI) pipelines. These checks run automatically on each change, acting as a gate during acceptance or merge. To examine whether experience shapes integration outcomes, we analyze how often agentic PRs from core and peripheral developers successfully pass all verification checks prior to acceptance. We find that peripheral developers are more likely to merge agentic PRs without running

Table 1: GitHub search queries to extract PRs following [52]

Agent	GitHub Search Query	Release Date
Claude Code	is:pr "Co-Authored-By: Claude"	2025-02-24
Cursor	is:pr head:cursor/	2025-01-01
GitHub Copilot	is:pr head:copilot/	2025-01-01
OpenAI Codex	is:pr head:codex/	2025-05-16

any checks, whereas core developers are more likely to ensure that accepted agentic PRs have passed the CI pipeline.

Replication package including the scripts and data to answer our RQs can be found in our online appendix [1].

2 Methodology

Figure 1 demonstrates the methodology of our study. The following sections provide a detailed description of each method.

2.1 Data Preparation

In this section, we describe the data preparation procedure.

Data Collection. We extracted PRs from GitHub associated with four widely adopted coding agents: Claude Code, Cursor, GitHub Copilot and OpenAI Codex. These agents were selected due to their increasing adoption in practice and their superior performance compared to other coding agents on the SWE-Bench¹. Data collection was conducted using the GitHub REST API² in combination with targeted search queries, following a methodology similar to Li et al. [52]. Table 1 presents the specific search queries employed to identify PRs linked to each agent. These queries were adapted from PRarena³ and leverage distinctive indicators such as branch name prefixes (e.g., "head:copilot/") or attribution statements in the PR body (e.g., "Co-Authored-By: Claude"). We further customized the search to include a specialized query for Claude Code and applied a start-date filter to remove unrelated PRs. Our dataset includes data up to August 1, 2025. Moreover, to ensure project relevance and data quality, we applied a filter to include only repositories with ≥ 100 GitHub stars, following the best practices for GitHub-based empirical analyses [43]. Finally, our dataset included 28,769 PRs, with 2,856 developers from 2,595 repositories. In the subsequent step, we describe how developer experience was measured.

Developer Experience. Prior literature establishes that a developer's project-specific experience is strongly correlated with their contribution volume [64, 70]. As developers contribute to a project's codebase through various activities, they transition from peripheral contributors to core maintainers [20]. Researchers have measured developers' experience to categorize them into core or peripheral using various contribution metrics. This includes the number of commits [27, 70], lines of code contributed [65], and developer seniority (i.e., years since first code contribution) [64]. Among these, commit count has been the most common proxy for developer experience [27].

However, commit-based measures have known limitations. Commits capture a single logical unit of work and are influenced by

¹<https://www.swebench.com/>

²<https://docs.github.com/en/rest?apiVersion=2022-11-28>

³<https://github.com/aavetis/PRarena>

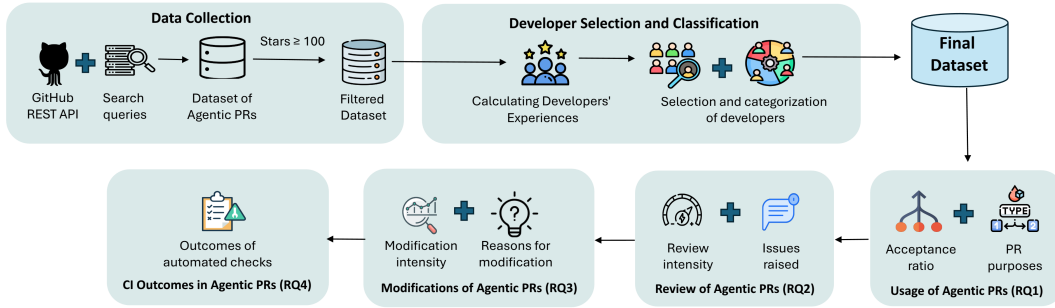


Figure 1: Overview of the methodology to investigate core and peripheral developers' use of coding agents

individual development styles [73]. For instance, one developer may commit frequently in small increments, whereas another may batch equivalent changes into fewer commits, leading to skewed experience estimates. Moreover, when developers transition to core roles, their commit activity often decreases as they shift toward coordination and review responsibilities [39].

Thus, prior work recommends estimating developer experience using a broad set of contribution activities [5, 73]. Following this guidance and recognizing pull-based development has become the dominant paradigm in open source software (OSS) projects [35], we adopt a PR-based metric to calculate developers' experience, as each PR encompasses both code commits and review [23]. We measured a developer's experience within a repository as the normalized sum of authored and reviewed closed PRs relative to the total number of closed PRs, following Kononenko et al. [47].

Developer selection. To retrieve a developer's activity, we used GitHub's REST API and their unique login identifiers. Next, we ensured that the experience metric only reflects preexisting expertise rather than PR activity induced by coding agents. Including developers whose contribution began after the introduction of agents would confound whether experience influenced agent usage or vice versa. Thus, we filtered out developers who had not authored any PRs before the introduction of agents (i.e., before January 2025 [52]). After this step, we identified 2,057 unique developers associated with 12,925 PRs and 1,899 repositories.

Since a developer can contribute to multiple repositories, experience scores may differ across projects. In prior work, developer roles are typically defined at the project level [14]. Thus, to avoid role ambiguity, where a developer could be core in one repository but peripheral in another, we further removed developers who collaborated on agentic PRs in more than one repository. Moreover, as our study targets completed development activities, we excluded all open PRs. Therefore, after applying these filtering steps, our final dataset consisted of 9,427 PRs from 1,391 repositories and 1,701 developers, whose activity was retrieved, and for whom we calculated their corresponding experience scores.

Developer Classification. We classify developers into two groups (i.e., core and peripheral) based on their computed experience scores. Prior research [20, 65] consistently shows that software projects exhibit skewed contribution patterns, where few developers account for the majority of contributions (i.e., approximately 80% [65]),

Table 2: An overview of the subject dataset

Item	Value
# of total closed/resolved agentic PRs	9,427
# of repositories	1,391
# of merged PRs (<i>accepted</i>)	6,819
# of developers	1,701
# of core developers	390
# of peripheral developers	1,311

while most contribute relatively little. In accordance, *core* developers are defined as those who contribute the most to a project, while the others are *periphery*. Following this established convention, we adopt the 80th percentile of the experience score distribution as the threshold for distinguishing core contributors from peripheral ones. With this criterion, we identify 390 developers as core contributors, while the remaining 1,311 developers are classified as peripheral contributors. Table 2 shows the descriptive statistics about our corpus after the data preparation steps have been applied.

2.2 Characterizing Agentic PR Usage

To answer RQ₁, we first measure and compute the frequency of accepted PRs among the 9,427 PRs across the core and peripheral groups of developers. We employ metrics and perform statistical analysis that have been previously applied in the literature [84].

Second, we aim to characterize the primary purpose of agentic PRs across different developer groups. Here, we limit our dataset to 6,819 accepted PRs to focus exclusively on contributions with the developer's intent to merge. Given the emerging stage of agent adoption and the low barrier to entry, developers explore agents' capabilities by prompting them to resolve hypothetical issues [81] rather than assigning actual issues. These exploratory PRs often lack substantive developer engagement, such as code reviews or follow-up modifications. For instance, in PR#699⁴, a developer prompted the Copilot agent to draft an experimental abstraction hierarchy but subsequently cancelled the operation after observing the initial output, and then closed the PR. Therefore, by focusing on accepted

⁴<https://github.com/Aiko-IT-Systems/DisCatSharp/pull/699>

PR, we isolate our study to PRs that resolve real-world issues within the project codebase.

Since a PR consists of one or more interrelated commits [55], we adopted the ten-category taxonomy proposed by Zeng et al. [96], which was originally developed for commit-level classification. However, we applied this taxonomy to characterize the purpose of agentic PRs, following a similar strategy to prior work [90]. The taxonomy categories are outlined in Table 3.

Table 3: Taxonomy used for categorizing PR purposes

Category	Description
fix	Bug and fault correction in the codebase.
feat	Code changes that introduce new functionalities, including internal or user-facing features.
refactor	code restructuring to improve maintainability without altering behavior.
docs	Updates documentation, or comments (e.g., README, API docs, typo fixes).
test	Adds or modifies test cases or test files.
build	Code changes that modify build systems or dependencies (e.g., Maven, Gradle, Cargo).
style	Enhances code readability or formatting without changing behavior.
ci	Updates CI/CD workflows and configuration files.
perf	Improves execution speed, memory usage, or efficiency.
chore	General maintenance tasks (e.g., dependency bumps, cleanup).

Note: We adopt the taxonomy from established prior work [96].

To characterize the purpose of the 6,819 accepted PRs, we employed the GPT-4 model to identify the PR categories from the taxonomy, following a recent SE study [49]. We selected GPT-4 due to its demonstrated capability in accurately annotating and categorizing SE artifacts [3]. To ensure reliability, we adopted a human validation approach consistent with prior work [49]. Specifically, we selected a random stratified sample of 364 PRs with proportionate strata (satisfying 95% confidence level with a 5% error rate). Here, proportionate stratification is appropriate because the goal is to validate the labeling accuracy of GPT-4 over the entire dataset, not to compare performance across strata. Then, we used GPT-4 to assign category labels from the selected taxonomy based on the PR title and body, allowing multiple labels per PR when applicable [90]. Next, the first two authors independently reviewed GPT-4's annotations for this subset, adding alternative labels where appropriate and documenting disagreements. Cases with inconsistent labeling were flagged for further review and underwent secondary evaluation with refinement in the prompting strategies [42]. The resulting inter-rater reliability (IRR), measured using Cohen's Kappa, was 80.3%, indicating strong agreement among annotators [60]. Given this validation and the higher observed alignment with GPT-generated labels, the remaining dataset was classified using GPT-4.

2.3 Analyzing Review Dynamics in Agentic PR

We aim to understand how rigorously agentic PRs are reviewed, as well as the patterns of issues raised during the review process by both core and peripheral developers, in order to address RQ2. Therefore, we analyze the 6,819 accepted PRs to identify those with review comments. We found 1,501 PRs containing review comments, resulting in 9,403 comments collected via GitHub's REST API endpoint⁵. We perform quantitative analysis on these comments to investigate the review intensity.

⁵<https://api.github.com/repos/{owner}/{repo}/pulls/{pr-number}/comments>

Table 4: Review comments taxonomy and description

Category	Description
🐛 Functional Defect	A functionality is missing or implemented incorrectly, which often requires additional code or major fixes.
🔍 Validation	Issues with invalid value detection and data sanitization.
🧠 Logical	Issues involving comparison, control flow, computation, or other logical errors.
🔌 Interface	Issues arising from external components such as libraries, hardware devices, database, or operating system.
🗄️ Resource	Issues related to the initialization, handling, or release of variables, memory, files, and databases.
🛠️ Support	Issues related to support systems, libraries, or their configurations.
⌚ Timing	Issues caused by improper thread synchronization when accessing shared resources.
💡 Solution Approach	Suggestions for alternate implementations (e.g., algorithms, data structures).
📄 Documentation	Suggestions to improve code comments or documentation.
🏗️ Organization of Code	Suggestions for structural refactoring, such as collapse hierarchy, extract superclass, and inline function.
🔄 Alternate Output	Suggestions for enhancing error messages, alerts, toast notifications, or function return values.
🏷️ Naming Convention	Suggestions for renaming software elements to comply with conventions.
👁️ Visual Representation	Suggestions for enhancing code readability, such as adjusting indentation, removing unnecessary whitespace, or reorganizing code.
❓ Question	Questions to understand design and implementation choices.
💬 Design Discussion	High-level discussion about design choices, design patterns, and software architecture.

Note: We adopt the code review comment categories and their descriptions from established prior work. [10, 57, 87]

🐛 Functional. 🔍 Evolvability. 🧠 Discussion

Next, to identify the patterns of issues raised by developers during these reviews, we apply a closed coding analysis [48]. We take the set of labels proposed by prior studies [10, 57] as our code booklet (Table 4). After this, we followed Tufano et al. [86] to preprocess the review comments. Specifically, we use the `code/Cleaner.py` script from their provided replication package. The script identifies non-English comments using multiple detectors and only retains comments classified as English. Furthermore, comments consisting solely of emojis, hyperlinks, acknowledgements (e.g., "LGTM", "+1), or conversational replies (e.g., "see above", "same here") are removed, as they do not convey actionable review intent. On top of that, we excluded comments authored by bots or agents, whose username contained the suffix "bot" [33], and accounts matching an established list of known bots or coding agents [34]. Subsequently, we also removed the comments made by other developers who were not present in our studied 1,701 developers. This resulted in a total of 2,735 comments from both developer groups.

Then, we employed stratified random sampling with equal strata, resulting in a total sample of 340 comments (satisfying a 95% confidence level and a $\pm 5\%$ margin of error). As our objective is to characterize patterns of each developer group independently rather than to analyze for the aggregate population, we applied group-specific sampling and analysis strategies following prior work [37]. The coding task was performed by the first two authors as coders. Initially, the coders independently labeled a separate set of 40 comments, achieving a Cohen's kappa coefficient of 0.71 (moderate agreement). Since this score was not sufficient to proceed, the authors convened to discuss the labeling. They addressed disputes and conflicts until reaching a consensus. After refining the labels, they re-labeled the 40 samples and confirmed their agreement. The IRR, measured by Cohen's Kappa, was found to be 84.5%, indicating near-perfect agreement [60]. Finally, the first two authors labeled the same set of comments individually and achieved a Cohen's Kappa score of 82.5%, signifying strong agreement [60].

2.4 Characterizing Agentic PR Modifications

To answer RQ₃, we filter the agentic PRs that were subsequently modified by developers from the total of 6,819 accepted PRs. We define an agentic PR as modified if a developer added one or more commits after the agent's initial submission. Although agents may also modify a PR in response to feedback, our study's focus is on understanding whether the degree and purpose of developer-driven modifications differ between core and peripheral contributors. Therefore, we included only those PRs in which the developer made subsequent commits following the agent's contribution. To do so, we first collect commit-level data using the GitHub REST API endpoint⁶ for all 6,819 accepted PRs. Next, we identify and distinguish developers' commits from the agent's commits within these PRs. When agents commit, they compose the commit message themselves and often advertise their presence in several ways: (1) By being the *author* of the commit, analogous to other bots (e.g., Dependabot). For example, commits by Copilot list the author as "copilot-swe-agent[bot]" with the login "Copilot". Similarly, commits by Cursor use the author name "Cursor Agent", the email "cursoragent@cursor.com" and the login "cursoragent". (2) By adding themselves as a *co-author* of the commit, with the agent's user recorded as the main author. For instance, commits by Claude used the "Co-authored-by:" trailer added. (3) By adding an additional trailer (such as *label*) when authoring a commit event. For example, a commit event by Codex had a "Codex" label added prior to it. Any commit that did not match the above patterns was classified as *developer-authored* commit. This filtering process enabled us to isolate 4,339 agent-assisted PRs where developer modifications were present. On these PRs, we measure and compare the degree of modifications between the developer groups based on lines of code added and deleted.

Then, to further understand the reasons behind these modifications, we conducted a closed coding analysis. As done in Section 2.3, we randomly sample 360 commit messages (satisfying a 95% confidence level with a $\pm 5\%$ margin of error). We used the commit-level taxonomy proposed by Zeng et al. [96] as our booklet. Following prior work [40, 66], commits with ambiguous descriptions were excluded and replaced with new samples to maintain the target sample size. The coding was performed by the first two authors, who had already established a shared understanding of labeling criteria through the earlier coding process (Section 2.2). Thus, both coders independently annotated the commits, achieving an IRR (Cohen's Kappa) of 81.9%, indicating strong agreement between annotators [60]. Any disagreements that occurred were resolved through discussions.

2.5 Analyzing CI Outcomes in Agentic PR

Modern Code Review (MCR) [63] is increasingly coupled with automated quality checks (i.e., static analysis, test execution, and build validation) [38]. Continuous integration (CI) features the automated execution of these checks every time a change is pushed. These self-testing safeguards aim to detect defects early and ensure that only secure and reliable changes enter a project's branch [29]. Importantly, all CI jobs should pass during the merge stage [79] and a previous study highlights that PRs with more passed checks have

a higher chance of being merged than broken CI [95]. Since agentic PRs are increasingly merged into the project codebase [52], we aim to examine how often these automated checks are bypassed by core and peripheral developers during merge commit, answering RQ₄.

In GitHub, whenever a new commit is added to a PR (by a developer or an agent), the platform automatically triggers `check_suite` event with an action of `requested` to all GitHub Apps installed by the project that have write permission. Note that a `check_suite` is a collection of the check runs created by a single CI App in GitHub, which summarizes the status and conclusion of the check runs. We use the GitHub REST API endpoint⁷ to collect the completed status of 6,819 accepted agentic PRs and compare the conclusion across the developer groups. When a check run completes, its conclusion can be *success*, *failure*, *neutral*, *cancelled*, *timed_out*, *skipped*, or *action_required*.

Based on the conclusion, we determine the rollup results of the CI checks, following a prior study [45]. That is, if all checks' conclusion is "success", then the rollup status of the CI is success, otherwise it is failure. Moreover, to ensure that the results accurately reflected the actions of our studied developers, we included only those PRs where the same developer both assigned the task to the agent and merged it. This filtering resulted in a final analysis set of 5,319 PRs from peripheral developers and 1,018 PRs from core developers to investigate the rollup results of CI checks.

3 Study Findings

We ask four research questions in this study. In this section, we answer them carefully with the help of our quantitative and qualitative findings as follows:

3.1 Usage Characteristics of Agentic PR (RQ₁)

Figure 2 provides a distribution of resolved agentic PRs across core and peripheral developers. Although the median number of agent usage per developer is identical across both groups (2.0), the mean is substantially higher for peripheral developers (6.08) compared to core developers (3.73). This suggests that while the *regular* level of AI delegation is similar, a subset of peripheral contributors generates disproportionately more agentic PRs. Our *Mann-Whitney Wilcoxon* [28] test also revealed a statistically significant difference ($p - value < 0.05$) between these two groups. However, the Cliff's Delta size is negligible ($|d| = 0.06$). These results suggest that, though statistically significant, the overall resolution patterns of agentic PRs between the two groups have minimal practical impact.

Moreover, as shown in Table 5, the overall acceptance rates of agentic PRs appear similar for core and peripheral developers. However, for the accepted PRs that were merged into the main/master branch, core developers exhibit a substantially higher merge rate than peripheral contributors. Prior research has shown that developer reputation within a repository can strongly influence merge outcomes [89], with PRs from core contributors being more likely to reach the main branch compared to those from peripheral developers [67].

Next, Figure 3 represents the underlying purposes of agentic PRs created by each developer group. Both core and peripheral developers use agents across all *ten* purpose types. However, agentic PRs

⁶<https://api.github.com/repos/{owner}/{repo}/commits/{commit-sha}>

⁷<https://api.github.com/repos/{owner}/{repo}/commits/{sha}/check-runs>

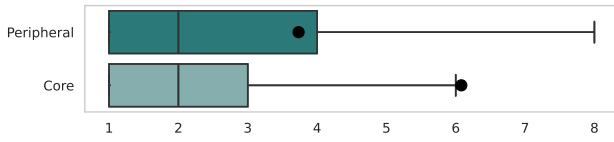


Figure 2: Distribution of resolved PRs across core and peripheral developers (• represents mean here)

Table 5: PR acceptance ratio across core and peripheral developers

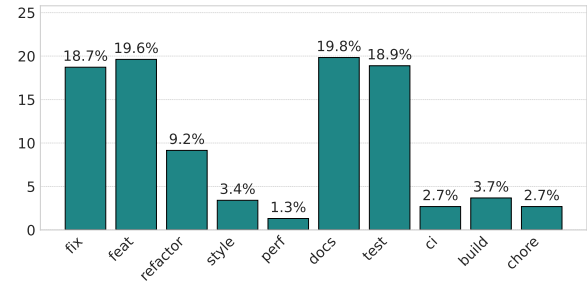
Developer Category	#Devs	#Resolved	#Accepted	Accepted(%)	$\mapsto m(\%)$
Peripheral	1,311	7,971	5,759	72.3	77.8
Core	390	1,456	1,060	72.8	85.8

“ $\mapsto m$ ” signifies PR been accepted or merged into the main/master branch.

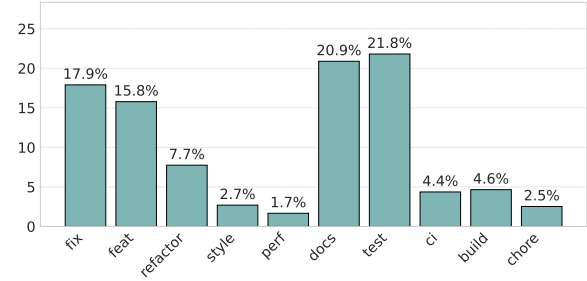
from both groups were mainly concentrated in *four* primary categories: *documentation* (*docs*), *testing* (*test*), *feature implementation* (*feat*), and *bug fixing* (*fix*). For core developers, these four categories accounted for 76.4% of their accepted agentic PRs, with the remaining PRs spread across *refactoring* (*refactor*), *non-functional adjustments* (*style*), *performance improvements* (*perf*), *CI/CD updates* (*ci*), *build configuration changes* (*build*), and *other cleanup activities* (*chore*). Peripheral developers show a similar pattern, where 77% of their agentic PRs fall within the four primary categories.

Moreover, **peripheral developers showed an even pattern of agent use across the four major PR types** (~18.7–19.8%). Our thematic analysis further reveals that peripheral developers’ activities showed a uniform AI delegation strategy rather than category-specific reliance. For example, in *PR#86*⁸, the developer delegated several tasks to the agent. The agent fixed multiple bugs in both the frontend and backend that were preventing the visualization system from loading and displaying data correctly. It also added new features such as clearer server messages. In addition, the agent updated documentation by adding new example scripts, and finally, it included changes related to testing and verification. In another *PR#174*⁹, the agent added a new feature to the HTTP function so users can view the redirect response content instead of automatically going to the next page. The agent also updated the documentation to show how to use this new option. These contributions illustrate how peripheral developers are leveraging agents to handle end-to-end maintenance as well as feature enhancement, reflecting a comprehensive approach to agent delegations.

Conversely, **core developers use agents mostly for documentation and testing tasks** (42.7% in total). In particular, 21.8% of core developers’ agentic PRs target testing tasks. For example, in *PR#12199*¹⁰, A developer asked the agent to add and test a new search feature that finds all two-factor authentication entries. The agent completed the task and also added tests to make sure the search works correctly, ignoring items without two-factor authentication, handling upper and lower case, and managing exclusion



(a) Peripheral developers



(b) Core developers

Figure 3: Purposes of agentic PRs across core and peripheral developers

searches properly. Core developers also frequently relied on agents for documentation updates (20.9%). In another *PR#386*¹¹, the developer assigned the agent to write a new documentation file that explains key details for developers, such as what the project is about, how to run commands, how to test the code, and what to check before submitting changes.

Lastly, core developers assign fewer agentic PRs to bug fixing (–3.8%) compared to peripheral developers. This pattern is similar to the studies involving human-authored PRs [2, 65], where peripheral developers act as the “many eyes” of the project and most frequently contribute small changes such as bug fixes.

We also investigate how the purposes of agentic PRs depend on the developer group. In particular, we identify two categorical variables - *PR types* and *developer groups*. We use *Chi-Squared* [61] statistical test to measure the independence between these two variables. We find a statistically significant *p-value* ($p\text{-value} \approx 0.0 < 0.0001$) from our analysis. Thus, there is a statistically significant association between developer group and PR type. This result indicates that the distribution of PR types differs significantly between core and peripheral developers.

RQ1 Summary: Both groups used agents at comparable rates, but a subgroup of peripheral developers relied on them disproportionately, and core developers achieved higher main/master-branch acceptance rates. Peripheral developers delegated tasks evenly across *bug fixing*, *feature addition*, *documentation*, and *testing*, whereas core developers focused mainly on *testing* and *documentation*.

⁸<https://github.com/Deep-Learning-Profiling-Tools/triton-viz/pull/86>

⁹<https://github.com/tomhrr/cosh/pull/174>

¹⁰<https://github.com/keepassxreboot/keepassxc/pull/12199>

¹¹<https://github.com/numbagg/numbagg/pull/386>

3.2 Review Dynamics of Agentic PRs (RQ₂)

Figure 4 summarizes the distribution of review comments across peripheral and core developers. The median number of comments per agentic PR is higher for core developers (3.6) compared to peripheral developers (2.0). To assess the statistical significance of this difference, we applied the *Mann–Whitney–Wilcoxon* test and found a statistically significant result ($p < 0.001$). The corresponding effect size, measured using *Cliff's Delta*, was small but non-negligible ($|d| = 0.28$). These findings suggest that core developers engage more thoroughly in review discussions on agentic PRs than peripheral developers, consistent with prior studies on human-authored PRs that identify core contributors as central actors in review [44].

Following this, Figure 5 presents the patterns of issues raised by

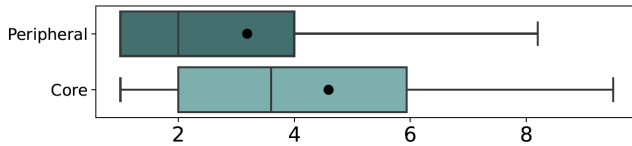


Figure 4: Distribution of reviews comments across peripheral and core developers (• represents mean here)

core and peripheral developers.

We found that the majority of comments from both groups focused on *evolvability*, aligning with prior observations in human-authored PRs [10]. Specifically, core developers raised *functional* (24.9%), *evolvability* (52.8%), and *discussion*-related issues (14.7%). Peripheral developers exhibited a similar distribution, with slightly fewer *functional* issues (18.3%), slightly more *evolvability* issues (59.3%), and a comparable proportion of *discussion*-related concerns (13.5%). Surprisingly, they did not raise any issues related to *resource* or *timing* types of functional issues in agentic PRs.

In addition, *code organization emerged as a prominent concern among peripheral developers* (17.6%). These review comments mainly focused on suggestions for structural refactoring (e.g., collapse hierarchy), an extra superclass, and an inline function. For example, in PR#41332¹², the reviewer suggested reorganizing the code by merging one section with another: “combine this section with the Message Handling Exceptions section”.

Whereas, *alternative solutions were frequently observed among core developers' review comments* (18.2%). Here, developers suggest different approaches, such as alternative algorithms or API choices. For example, in PR#22840¹³, the reviewer proposed an alternative implementation by suggesting the use of a more suitable method across the file to improve consistency and correctness: “Use GetIncludedPlatforms instead of GetAllPlatforms in all cases in this file.”. Lastly, both peripheral and core developers frequently seek clarification on design rationale or implementation choices from the agents. We also use the Chi-Squared test to measure the independence of these two categorical variables - comment category and developer group. However, we did not find any statistically significant association between these two variables.

¹²<https://github.com/Azure/azure-sdk-for-python/pull/41352>

¹³<https://github.com/dotnet/macios/pull/22840>

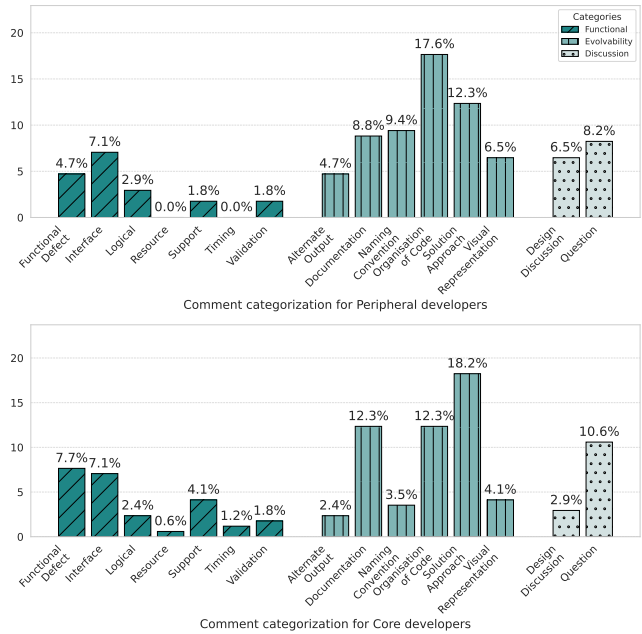


Figure 5: Prevalence of code review comments across developer experience groups

RQ₂ Summary: Core developers engage slightly more in review discussions than peripheral developers. Both groups raise *evolvability* types of review issues largely, with peripheral developers tending to emphasize *code organization* concerns, and core developers more frequently focused on providing *alternative solutions* to the agents.

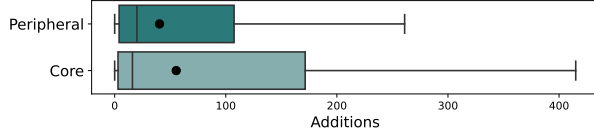
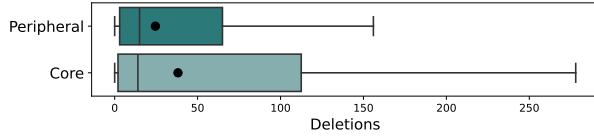
3.3 Modifications of Agentic PRs (RQ₃)

Table 6 provides the summary of PRs that were modified by core and peripheral developers after the agent's initial submission and before final acceptance. In most cases, the agent's contribution was accepted *without modification* with an average of 74.1% across both developer groups. However, core developers modified a larger share of agentic PRs, representing 28.3% of their 1,060 accepted PRs, compared to 23.5% of the 5,759 PRs accepted by peripheral developers.

In addition, Figure 6 shows that the median number of added and deleted lines remains small for both groups, indicating that developers typically make minor adjustments rather than rewriting the agent's contribution. However, core developers show a higher mean percentage of added lines of code at 55.4% compared to 40.4% for peripheral developers. They also have a greater percentage of deleted lines of code, with 38.2% versus 24.5% for peripheral developers. The wider spread in core developers' distributions also suggests that, while most modifications are modest, a subset of core developers performs substantially larger edits on agent-generated code. Despite this difference in spread, the *Mann–Whitney–Wilcoxon* test did not reveal a statistically significant difference between the two groups.

Table 6: Dynamics of Agentic PR modifications

Developer Category	#Accepted	#Modified	Mod. No Mod. (%)	LOC (added)	LOC (deleted)
Peripheral	5759	1353	23.5 76.5	40.4	24.5
Core	1060	300	28.3 71.5	55.4	38.2

**(a) Lines of code added by developers****(b) Lines of code deleted by developers****Figure 6: Added and deleted lines in agentic PR modifications across developer groups (• represents mean here)**

Subsequently, on our thematic analysis of developers' commit messages, Figure 7 summarizes the reasons behind the modifications to agents' code.

Peripheral contributors frequently modify the agent's code to fix bugs and refactor (36.2% in total). For example, in *PR#4532*¹⁴, the agent was assigned to check a model's responses, which produced short, blocked, or unusable answers. To address this, the agent updated the code with an unblocking step that automatically reprocesses such replies. However, when some test cases failed, the developer modified the agent by adding 220 lines and deleting 294 lines of code by fixing type errors, adding safety checks, and fixing the logic based on feedback. In another example, in *PR#1532*¹⁵, the agent reorganized how debugging messages were handled by moving scattered logging code into a shared location for easier reuse and maintenance. After a reviewer noticed a missing version update that could break the code, the developer added 7 lines of code to fix the issue by updating the version number.

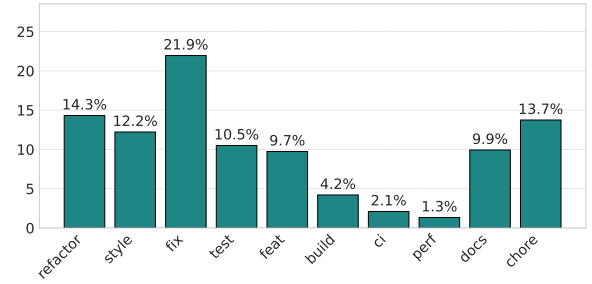
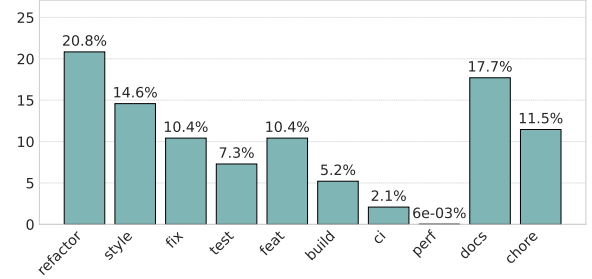
Core developers mainly refactored the agent's contribution and improved its documentation (38.5% in total). For instance, in *PR#2801*¹⁶, the agent authored eight commits focusing on big structural tasks, such as moving files, reorganizing components, and cleaning up utilities. These were large-scale, automatic changes that reshaped the project, but the contribution failed to pass all the checks. Then, the core developer made 19 commits (LOC added: 1154 and LOC deleted: 1126) afterward. He fixed type errors, cleaned up code style, adjusted imports and localization and ensured the CI tests pass. He also reverted one of the Copilot's commits where he manually undid part of the agent's code because it caused issues or was not needed. In another *PR#107*¹⁷, the agent was asked to

¹⁴<https://github.com/promptfoo/promptfoo/pull/4532>

¹⁵<https://github.com/LayerZero-Labs/devtools/pull/1532>

¹⁶<https://github.com/polkadot-cloud/polkadot-staking-dashboard/pull/2801>

¹⁷<https://github.com/PythonNest/PyNest/pull/107>

**(a) Peripheral developers****(b) Core developers****Figure 7: Categorization of commits for peripheral and core developers**

add a new feature that helps control who can access different parts of a system. The developer improved the explanation by adding a total of 83 lines, including clear, real-world examples showing how the new feature could be used, making it easier for others to understand and apply.

Lastly, we observe that both developer groups make very few modifications to agent-generated code related to build, CI, and performance. This is likely because agents are assigned to these tasks less frequently, as shown in our result Section 3.1. We further applied the Chi-Squared test to measure the independence of these two categorical variables - modification category and developer group. However, the test did not reveal a statistically significant association.

RQ₃ Summary: On average, 74.1% of agentic PRs were accepted without modification. When revisions did occur, core and peripheral developers modified agent-generated code in broadly similar ways. Notably, both groups frequently engaged in *refactoring*, while peripheral developers additionally addressed *bug fixes*, and core developers also focused on *documentation improvements*.

3.4 CI outcomes of Agentic PRs (RQ₄)

Table 7 shows that core contributors are more likely to merge agentic PRs that meet CI requirements compared to peripheral developers. On average, core contributors run slightly more checks per agentic PR than peripheral developers, with averages of 9.3 and 8.1 check runs, respectively. Additionally, core developers achieve a higher proportion of "success" outcomes in all the check_suites added to the project when compared to peripheral developers, with

Table 7: Summary of CI check on Agentic PRs

Developer Category	Checks	Success (%)	Failure (%)	No Check (%)
Peripheral	8.1	43.1	20.0	19.1
Core	9.3	51.2	16.5	11.2

rates of 51.2% and 43.1%, respectively. In contrast, peripheral developers are nearly twice as likely to merge without running any checks on the agentic PR (19.1% vs. 11.2%).

For example, in *PR#34521*¹⁸, a peripheral contributor asked Copilot to migrate a set of UI tests. After Copilot's first commit failed the CI pipeline, the developer replied: *"CI build is failing."* Copilot pushed another commit and replied: *"I've resolved the issue [...] all tests are now passing,"* but the CI still failed. The developer then offered minimal guidance, and after two more Copilot attempts, again responded: *"The tests are failing."* After these back and forths, Copilot updated the tests to align with the component's behavior, and the CI passed. Finally, the developer accepted the PR with two integration checks still failing.

Whereas, in *PR#103*¹⁹, a core developer tasked Copilot with fixing a minor usability issue (i.e., ensuring that the UI would remember the user's selected view mode instead of resetting after a page reload). Copilot's initial commit successfully addressed the functionality, but when the project's dedicated checks ran, the change failed the prettier formatting check. This indicated that Copilot did not fully adhere to the project's code style guidelines. The core developer then corrected the formatting in a follow-up commit and cleaned up the tests written by Copilot, after which all CI checks passed, and the PR was merged into the main branch.

These results indicate that core developers more consistently adhere to the *"all checks must pass"* principle emphasized in CI literature [4, 79], whereas peripheral contributors are more willing to merge despite incomplete verification of an agentic PR.

RQ4 Summary: Peripheral developers are more likely to merge agentic PRs without executing verification checks, whereas core developers more consistently require successful CI results on acceptance.

4 Lessons Learned and Implications

In this section, we distill six lessons for the community and propose the following implications based on them.

What's old is new. Our findings (Section 3.1) show that even though the number of accepted agentic PRs is similar for core and peripheral contributors, core contributors' agentic PRs are accepted into the main/master branch at a much higher rate. This suggests that socio-technical factors, such as contributor reputation [89], which previously influenced the likelihood of merging human-authored PRs into a project's main branch [67, 98], similarly affect agentic PRs. In other words, gatekeeping at the main branch persists even when both groups delegate work to agents; the human social layer still mediates agent outputs. However, we argue that *AI-assisted workflows should be shaped to reduce, not amplify, such*

long-standing socio-technical inequities [32]. Concretely, the OSS community can establish standardized review rubrics and checklists, ensuring that evaluations are based on the agent's contributions rather than the contributor's socio-technical factors.

Agents reducing the time spent on "toil". Our results (Section 3.1) show that when developers delegate work to agents, they most often offload toil-heavy tasks (e.g., documentation, testing). Prior studies [49, 76] note that developers often view testing as tedious and would prefer to spend less time on it, and the 2024 Stack Overflow survey²⁰ highlights documentation as a key area where developers expect the greatest growth in AI assistance. These activities are disproportionately seen as work to minimize, positioning them as strong candidates for AI support. Our empirical results confirm that this shift is already underway in real-world OSS projects: *both core and peripheral contributors predominantly delegate these toil-heavy tasks to agents*, underscoring how AI is being used to absorb low-satisfaction work in practice.

These findings also point to a practical opportunity for enterprise organizations, especially those with stricter AI usage policies [75], *to make better use of coding agents for toil*. Companies can support this by educating developers about the strengths and limitations of these tools and by setting clear usage guidelines. As highlighted in Google's DORA 2025 report²¹, effective adoption requires more than simply adding AI to the workflow; it also depends on building AI literacy and establishing strong governance practices.

Towards cultivating more trust. In Section 3.3, we show that 74.1% of agentic PRs are accepted without any developer intervention, consistent with prior findings [90]. However, Section 3.1 reveals that these successes are concentrated in routine tasks such as documentation and testing, whereas developers rarely rely on agents for complex activities like performance tuning or build changes.

Prior research shows that developers benefit when they understand when AI is likely to perform well [16], and that unclear reliability on complex tasks can miscalibrate trust and reduce productivity [68]. These findings point to a design opportunity for platforms like GitHub: *making agents' competence visible at the moment of use*. For instance, platforms could present confidence indicators based on usage statistics, historical performance, or feedback for similar task types, helping developers decide when to trust, verify, or ignore an agent's output.

Risk of peripherals becoming "power-users." Our results (see Section 3.1) indicate that a subset of peripheral contributors drives a significant portion of the agentic flow. They may be using AI to compensate for skill gaps [19] through autonomous agents that submit PRs on their behalf. While this can enable newcomers to contribute more quickly, it raises concerns about an over-reliance on agent-generated PRs, which could hinder learning and create an illusion of progress [13].

Research [59] shows that newcomers who do not engage deeply with the changes they submit risk struggling to develop long-term skills. Therefore, to promote sustainable learning, OSS communities could encourage newcomers to start with human-authored contributions before transitioning to more agentic workflows. This

¹⁸<https://github.com/microsoft/fluentui/pull/34521>

¹⁹<https://github.com/jamdotdev/jam-dev-utilities/pull/103>

²⁰<https://survey.stackoverflow.co/2024/ai#developer-tools-ai-next>

²¹<https://blog.google/technology/developers/dora-report-2025/>

approach would ensure that *automation enhances, rather than replaces, developer growth*.

Impact of “Heroes” on AI-assisted software development. In Section 3.3, we observe that a small subset of core contributors (“heroes”) [2] make substantially larger edits to agent-generated changes before merge. Most of these edits from heroes involve refactoring, which aligns with prior work showing that senior developers focus on maintainability and architecture [94].

Moreover, multiple recent studies also caution that AI-generated code can break quality or security standards, especially in complex projects [30, 54]. This means core contributors’ oversight and careful review are more necessary now before merging agent-generated code to maintain project quality.

Core developers remain “quality gatekeepers”. Our results from Section 3.4 show that core developers more consistently follow the *all-checks-must-pass* principle and achieve higher “success” rates, while peripheral developers are more likely to merge agentic PRs despite incomplete verification. These patterns align with prior work showing that core developers act as organization quality gatekeepers in agentic contribution as well [31]. They enforce CI compliance and iterative validation more rigorously than peripheral developers.

As automation and AI-assistance are increasingly handling testing and build verification, as we also saw in Section 3.1, such oversight by core developers becomes pivotal for maintaining accountability and release confidence [50]. Our findings suggest that teams should promote stricter validation before merging agentic code. For example, adopting *monthly audit* policies and *quality-gate-reviews* led by core developers can safeguard further improve the reliability and maintainability of CI pipelines.

5 Related Works

Recent research highlights rapid growth in human–AI collaboration across software engineering tasks such as code search [53], code generation [22], and automated program repair [92], with multiple studies demonstrating measurable productivity gains. For instance, Ziegler et al. [99] found that Copilot improves developer productivity across all skill levels, while other studies further confirm that integrating AI-powered tools into development workflows enhances overall efficiency [8, 82]. However, subsequent research has revealed limitations in these tools’ effectiveness and their impact on developer cognition [72, 88]. For example, Prather et al. [72] found that generative AI can amplify the metacognitive difficulties of novice programmers, while Vaithilingam et al. [88] reported that AI tools do not always improve completion time or success rate and often produce uninterpretable or buggy code.

The impact of AI programming assistants also varies across developer experience levels, with distinct usage patterns [6, 51]. Some studies suggest that less-experienced developers gain greater productivity benefits [21], while others highlight how AI assistance supports learning by reducing cognitive load and improving usability [6, 58]. In contrast, experienced developers often face challenges such as cognitive interruption and frustration when adapting AI-generated suggestions [36, 71]. These findings underscore that developer experience fundamentally shapes how AI assistance is used and perceived.

As autonomous coding agents enter the software development landscape, accompanied by bold claims about their potential to replace human developers [78], research has begun to shift from studying tool capabilities to understanding collaboration dynamics. Terragni et al. [83] envisioned a symbiotic partnership between human developers and AI systems, while Hassan et al. [3] framed this transition as Software Engineering 3.0, where agents act as intelligent collaborators rather than passive assistants. Yet, empirical evidence remains limited, such as Takerngsaksiri et al. [82] found that agents perform well under clearly defined specifications but struggle in agile, conversational workflows. GitHub-based studies [52, 90] similarly report growing adoption of coding agents, though their PRs are less likely to be accepted for complex tasks. Beyond copilots, developers have also reported challenges with proactive AI systems that act autonomously, often taking unsolicited actions or overwhelming users with excessive information [26]. Recent work on steering agentic tools [25] emphasizes that effective human–agent collaboration requires contextual awareness, transparency, and robust feedback mechanisms.

Together, these findings reveal a paradigm shift, from developers being assisted by AI to collaborating with autonomous coding agents as active teammates. Yet, empirical evidence is lacking on how developers with different experience levels interact with such agents in real-world software development settings. This study addresses that gap by examining how developers of varying experience levels delegate, review, modify, and validate agent-generated code. Understanding these experience-driven collaboration patterns is crucial for designing trustworthy, effective, and sustainable human–AI workflows in modern software development.

6 Threats to Validity

Below, we discuss the threats to the validity of our study.

External validity. This study focuses on four coding agents, i.e., Claude Code, OpenAI Codex, Copilot, and Cursor. Therefore, the findings may not generalize to all autonomous coding agents or future versions with different interaction capabilities. Moreover, as our analysis is restricted to GitHub-hosted projects, the observed collaboration patterns may not fully represent behaviors in other development environments (e.g., GitLab, Bitbucket, or proprietary enterprise workflows).

Moreover, we filtered repositories by a minimum threshold of 100 GitHub stars, which may bias our sample toward mature and popular projects. As a result, the findings may not generalize to smaller or emerging repositories, where collaboration practices and agent adoption behaviors could differ. Lastly, our dataset includes only developers active after agent introduction, potentially limiting generalization to the broader developer population, including multi-repository contributors and newcomers who may interact with agents differently. However, we deliberately applied these filters to ensure that our experience metric reflects preexisting, project-level expertise rather than experience inflated by agent-assisted activity.

Internal Validity. Our manual coding analyses introduce the subjectiveness of the inspectors to our results. To mitigate this, we follow common practices for all of our manual analyses [48, 56]. Every disagreement is discussed in a team of two authors until consensus is reached. The coders achieve Cohen’s Kappa scores of

80.3%–82.5% for the manual coding tasks. For PR purpose labeling, we employed GPT-4 for large-scale annotation. While automated classification may introduce labeling errors, we mitigated this by manually validating 364 PRs, achieving a Cohen's Kappa of 80.3%, indicating substantial agreement.

A further threat concerns our measurement of developer experience, which is based on review participation within a single repository. Although this may overlook experience from other projects, we restricted our scope to ensure consistency when examining repository-specific collaboration with coding agents. Our count-based metric may misclassify a small number of contributors as core or peripheral. However, given the large size of both groups, such errors would have a negligible impact, and we do not expect any systematic impact in our results.

Developers may modify agent-generated code through force pushes, which overwrite commit history. Because these edits leave no observable record, some developer interventions may not be captured in our analysis. Additionally, we applied the heuristics to distinguish human commits from agent-generated commits according to the current practice at the time of our data collection. Future studies should extend and refine these detection heuristics. Finally, to calculate CI outcomes, we relied on the `conclusion` field from the GitHub REST API. This introduces a potential threat to validity because some PRs may be merged even when checks fail, for example, if test failures are unrelated to the submitted changes. As a result, CI status may not always perfectly reflect the true integration of the corresponding agentic PRs.

Construct validity. We use the *Mann-Whitney-Wilcoxon* test, which is a widely used non-parametric test for evaluating the difference between two sample sets. However, the significance level might suffer due to the limited size of the samples. We thus consider the effect size along with p - *value*. To examine the association between two categorical variables, we also use the *Chi-Squared* test. This statistical test of independence is appropriate when the number of categories is small (≤ 20) [61]. Thus, threats to construct validity might be mitigated.

7 Conclusion and Future Work

This study provides the first empirical evidence of how developers with different experience levels collaborate with autonomous coding agents in real-world software projects. Our analysis of 6,819 accepted agentic PRs shows that, although developers exhibit similar acceptance behaviors, their interaction patterns vary by experience. Core developers primarily delegate documentation and testing tasks to agents, engage more deeply in reviews by suggesting alternative solutions, focus on refactoring for maintainability, and achieve higher merge success. In contrast, peripheral developers use agents across broader task categories, emphasize code organization in reviews, mainly fix bugs in agent-generated code, and tend to merge PRs even when verification is incomplete. These findings highlight that developer experience continues to shape collaboration patterns in the era of agentic software development. Understanding these dynamics can develop better tools, workflows, and training strategies to foster more effective and trustworthy human-AI collaboration.

For future work, we plan to extend this study by examining rejected agentic PRs, exploring longitudinal changes in developer-agent collaboration, and conducting qualitative interviews to better understand how developers perceive trust, accountability, and control when working with autonomous agents. We also aim to investigate how prompt design, feedback mechanisms, and organizational factors influence the success and reliability of agentic contributions.

Acknowledgments

This research is supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grants program, the Canada Foundation for Innovation's John R. Evans Leaders Fund (CFI-JELF), and by the industry-stream NSERC CRE-ATE in Software Analytics Research (SOAR).

References

- [1] Shamse Tasnim Cynthia, Joy Krishan Das, and Banani Roy. 2025. Replication Package of Are We All Using Agents the Same Way? An Empirical Study of Core and Peripheral Developers' Use of Coding Agents. <https://github.com/cynthia247/MSR-2026/>
- [2] Amritanshu Agrawal, Akond Rahman, Rahul Krishna, Alexander Sobran, and Tim Menzies. 2018. We don't need another hero? the impact of "heroes" on software development. In *Proceedings of the 40th international conference on software engineering: software engineering in practice*. 245–253.
- [3] Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2025. Can llms replace manual annotation of software engineering artifacts? In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 526–538.
- [4] Mohammed Shamsul Arefeen and Michael Schiller. 2019. Continuous integration using GitLab. *Undergraduate Research in Natural and Clinical Science and Technology Journal* 3 (2019), 1–6.
- [5] Linda Argote and Ella Miron-Spektor. 2011. Organizational learning: From experience to knowledge. *Organization science* 22, 5 (2011), 1123–1137.
- [6] Yasya Rusyda Aslina and I Gusti Bagus Baskara Nugraha. 2024. Exploring Potential AI Use Cases in Internal Developer Portals: A Path to Enhanced Developer Experience. In *2024 IEEE International Conference on Data and Software Engineering (ICoDSE)*. IEEE, 143–148.
- [7] Tae Hyun Baek and Minseong Kim. 2023. Is ChatGPT scary good? How user motivations affect creepiness and trust in generative artificial intelligence. *Telematics and Informatics* 83 (2023), 102030.
- [8] Leonardo Banh, Florian Holldack, and Gero Strobel. 2025. Copiloting the future: How generative AI transforms Software Engineering. *Information and Software Technology* 183 (2025), 107751.
- [9] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [10] Moritz Beller, Alberto Bacchelli, Andy Zaidman, and Elmar Juergens. 2014. Modern code reviews in open-source projects: Which problems do they fix?. In *Proceedings of the 11th working conference on mining software repositories*. 202–211.
- [11] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2022. Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20, 6 (2022), 35–57.
- [12] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [13] Erik Brynjolfsson, Daniel Rock, and Chad Syverson. 2017. *Artificial intelligence and the modern productivity paradox: A clash of expectations and statistics*. Technical Report. National Bureau of Economic Research.
- [14] Denivan Campos, Luana Martins, and Ivan Machado. 2022. An empirical study on the influence of developers' experience on software test code quality. In *Proceedings of the XXI Brazilian Symposium on Software Quality*. 1–10.
- [15] Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. 2025. Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. *arXiv preprint arXiv:2507.08149* (2025).
- [16] Rudrajit Choudhuri, Bianca Trinkenreich, Rahul Pandita, Eirini Kalliamvakou, Igor Steinmacher, Marco Gerosa, Christopher Sanchez, and Anita Sarma. 2025. What Needs Attention? Prioritizing Drivers of Developers' Trust and Adoption of Generative AI. *arXiv preprint arXiv:2505.17418* (2025).
- [17] Lena Chretien. 2025. We Analyzed 146,000 Jira tickets for Copilot Users. Here's What We Found. - Jellyfish - jellyfish.co/blog/we-analyzed-

- 146000-jira-tickets-for-copilot-users-heres-what-we-found/.
- [18] McKinsey & Company. 2023. Unleashing developer productivity with generative AI. <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai>.
 - [19] Kevin Crowston and Francesco Bolici. 2025. Deskillling and upskilling with AI systems. *Information Research an international electronic journal* 30, iConf (2025), 1009–1023.
 - [20] Kevin Crowston, Kangning Wei, Qing Li, and James Howison. 2006. Core and periphery in free/libre and open source software team communications. In *Proceedings of the 39th annual Hawaii international conference on system sciences (HICSS'06)*, Vol. 6. IEEE, 118a–118a.
 - [21] Zheyuan Kevin Cui, Mert Demirel, Sonia Jaffe, Leon Musolf, Sida Peng, and Tobias Salz. 2025. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. Available at SSRN 4945566 (2025).
 - [22] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
 - [23] Tapajit Dey and Audris Mockus. 2020. Effect of technical and social factors on pull request quality for the npm ecosystem. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 1–11.
 - [24] Thomas Dohmke. 2022. GitHub Copilot is generally available to all developers. <https://github.blog/news-insights/product-news/github-copilot-is-generally-available-to-all-developers/>.
 - [25] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkan Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent AI systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 1–15.
 - [26] Linda Erlenhov, Francisco Gomes De Oliveira Neto, and Philipp Leitner. 2020. An empirical study of bots in software development: Characteristics and challenges from a practitioner's perspective. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 445–455.
 - [27] Jon Eyolfson, Lin Tan, and Patrick Lam. 2011. Do time of day and developer experience affect commit bugginess?. In *Proceedings of the 8th Working Conference on Mining Software Repositories*, 153–162.
 - [28] Michael P Fay and Michael A Proschan. 2010. Wilcoxon-Mann-Whitney or t-test? On assumptions for hypothesis tests and multiple interpretations of decision rules. *Statistics surveys* 4 (2010), 1.
 - [29] Martin Fowler and Matthew Fommel. 2006. Continuous integration.
 - [30] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security weaknesses of copilot-generated code in github projects: An empirical study. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–34.
 - [31] Keheliya Gallaba. 2019. Improving the robustness and efficiency of continuous integration and deployment. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 619–623.
 - [32] Yan Gao and Yan Cui. 2020. Deep transfer learning for reducing health care disparities arising from biomedical data inequality. *Nature communications* 11, 1 (2020), 5131.
 - [33] Mehdi Golzadeh, Alexandre Decan, and Natarajan Chidambaram. 2022. On the accuracy of bot detection techniques. In *Proceedings of the Fourth International Workshop on Bots in Software Engineering*, 1–5.
 - [34] Mehdi Golzadeh, Alexandre Decan, Damien Legay, and Tom Mens. 2021. A ground-truth dataset and classification model for detecting bots in GitHub issue and PR comments. *Journal of Systems and Software* 175 (2021), 110911.
 - [35] Georgios Gousios, Martin Pinzger, and Arie van Deursen. 2014. An exploratory study of the pull-based software development model. In *Proceedings of the 36th international conference on software engineering*, 345–355.
 - [36] Emanuela Guglielmi, Venera Arnoudova, Gabriele Bavota, Rocco Oliveto, and Simone Scalabrino. 2025. How do Copilot Suggestions Impact Developers' Frustration and Productivity? *arXiv preprint arXiv:2504.06808* (2025).
 - [37] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
 - [38] Michael Hilton, Nicholas Nelson, Timothy Tunnell, Darko Marinov, and Danny Dig. 2017. Trade-offs in continuous integration: assurance, security, and flexibility. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 197–207.
 - [39] Corey Jergensen, Anita Sarma, and Patrick Wagstrom. 2011. The onion patch: migration in open source ecosystems. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, 70–80.
 - [40] Jing Jiang, Jiangfeng Lv, Jiateng Zheng, and Li Zhang. 2021. How developers modify pull requests in code review. *IEEE Transactions on Reliability* 71, 3 (2021), 1325–1339.
 - [41] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
 - [42] Aditya Kahol, Anka Chandras Tummepalli, and Preethu Rose Anish. 2025. OSS-LCAF: Open-Source Software License Conflict Analysis Framework. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 303–314.
 - [43] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M German, and Daniela Damian. 2014. The promises and perils of mining github. In *Proceedings of the 11th working conference on mining software repositories*, 92–101.
 - [44] Noureddine Kerzazi and Ikram El Asri. 2016. Who can help to review this piece of code?. In *Working Conference on Virtual Enterprises*. Springer, 289–301.
 - [45] Ali Khatami and Andy Zaidman. 2024. State-of-the-practice in quality assurance in Java-based open source software development. *Software: Practice and Experience* 54, 8 (2024), 1408–1446.
 - [46] Jin W Kim, Hye I Jo, and Bong G Lee. 2019. The study on the factors influencing on the behavioral intention of chatbot service for the financial sector: Focusing on the UTAUT model. *Journal of Digital Contents Society* 20, 1 (2019), 41–50.
 - [47] Oleksii Kononenko, Olga Baysal, and Michael W Godfrey. 2016. Code review quality: How developers see it. In *Proceedings of the 38th international conference on software engineering*, 1028–1038.
 - [48] Klaus Krippendorff. 2018. *Content analysis: An introduction to its methodology*. Sage publications.
 - [49] Sukrit Kumar, Drishti Goel, Thomas Zimmermann, Brian Houck, B Ashok, and Chetan Bansal. 2025. Time Warp: The Gap Between Developers' Ideal vs Actual Workweeks in an AI-Driven Era. *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (2025).
 - [50] E. Laukkanen, M. Paasivaara, Juha Itkonen, and C. Lassenius. 2018. Comparison of release engineering practices in a large mature company and a startup. *Empirical Software Engineering* 23 (2018), 3535 – 3577. <https://api.semanticscholar.org/CorpusId:4450768>
 - [51] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing code explanations created by students and large language models. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, 124–130.
 - [52] Hao Li, Haoxiang Zhang, and Ahmed E Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv preprint arXiv:2507.15003* (2025).
 - [53] Chao Liu, Xindong Zhang, Hongyu Zhang, Zhiyuan Wan, Zhan Huang, and Meng Yan. 2024. An empirical study of code search in intelligent coding assistant: Perceptions, expectations, and directions. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 283–293.
 - [54] Yue Liu, Thanh Le-Cong, Ratnadira Widayarsi, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D Le, and David Lo. 2024. Refining chatgpt-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–26.
 - [55] Zhongxin Liu, Xin Xia, Christoph Treude, David Lo, and Shanping Li. 2019. Automatic generation of pull request descriptions. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 176–188.
 - [56] Evanthis Lyons and Adrian Coyle. 2021. *Analysing qualitative data in psychology*. Sage Publications.
 - [57] Mika V Mäntylä and Casper Lassenius. 2008. What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering* 35, 3 (2008), 430–448.
 - [58] Sai Veda Prakash Masetty and Sreeja Vallamulla. 2025. Enhancing Novice Programming Education through AI-Driven Mentorship and Project-Based Learning. In *2025 IEEE Integrated STEM Education Conference (ISEC)*. IEEE, 1–7.
 - [59] Zainab Masood, Rashina Hoda, Kelly Blincoe, and Daniela Damian. 2022. Like, dislike, or just do it? How developers approach software development tasks. *Information and Software Technology* 150 (2022), 106963.
 - [60] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.
 - [61] Mary L McHugh. 2013. The chi-square test of independence. *Biochemia medica* 23, 2 (2013), 143–149.
 - [62] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E Hassan. 2014. The impact of code review coverage and code review participation on software quality: A case study of the qt, vtk, and itk projects. In *Proceedings of the 11th working conference on mining software repositories*, 192–201.
 - [63] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E Hassan. 2016. An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering* 21, 5 (2016), 2146–2189.
 - [64] Audris Mockus. 2010. Organizational volatility and its effects on software defects. In *Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering*, 117–126.

- [65] Audris Mockus, Roy T Fielding, and James D Herbsleb. 2002. Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 11, 3 (2002), 309–346.
- [66] Mahtab Nejati, Mahmoud Alfadél, and Shane McIntosh. 2023. Code review of build system specifications: Prevalence, purposes, patterns, and perceptions. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1213–1224.
- [67] Marco Ortu, Giuseppe Destefanis, Daniel Graziotin, Michele Marchesi, and Roberto Tonelli. 2020. How do you propose your code changes? Empirical analysis of affect metrics of pull requests on github. *IEEE access* 8 (2020), 110897–110907.
- [68] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [69] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of AI on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [70] Daryl Posnett, Raissa D'Souza, Premkumar Devanbu, and Vladimir Filkov. 2013. Dual ecological measures of focus in software development. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 452–461.
- [71] James Prather, Brent N Reeves, Paul Denny, Brett A Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's weird that it knows what i want": Usability and interactions with copilot for novice programmers. *ACM transactions on computer-human interaction* 31, 1 (2023), 1–31.
- [72] James Prather, Brent N Reeves, Juho Leinonen, Stephen MacNeil, Arisosa S Randrianasolo, Brett A Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. 2024. The widening gap: The benefits and harms of generative AI for novice programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research-Volume 1*. 469–486.
- [73] Romain Robbes and David Röthlisberger. 2013. Using developer interaction data to compare expertise metrics. In *2013 10th Working Conference on Mining Software Repositories (MSR)*. IEEE, 297–300.
- [74] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating agent-based program repair at google. *arXiv preprint arXiv:2501.07531* (2025).
- [75] Daniel Schiff, Justin Biddle, Jason Borenstein, and Kelly Laas. 2020. What's next for AI ethics, policy, and governance? a global overview. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. 153–158.
- [76] Agnia Sergeev, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (2025), 107610.
- [77] Jagdish N Sheth, Bruce I Newman, and Barbara L Gross. 1991. Why we buy what we buy: A theory of consumption values. *Journal of business research* 22, 2 (1991), 159–170.
- [78] Sherin Shibu. 2025. 'I do have a fair amount of concern': The CEO of \$61 billion Anthropic says AI will take over a crucial part of software engineers' jobs within a year. <https://www.entrepreneur.com/business-news/anthropic-ceo-predicts-ai-will-take-over-coding-in-12-months/488533>. Accessed: 2025-10-13.
- [79] Eliezio Soares, Gustavo Sizilio, Jadson Santos, Daniel Alencar Da Costa, and Uirá Kulesza. 2022. The effects of continuous integration on software development: a systematic literature review. *Empirical Software Engineering* 27, 3 (2022), 78.
- [80] Viktoria Stray, Nils Brede Moe, Nivethika Ganeshan, and Simon Kobbenes. 2025. Generative AI and developer workflows: How github Copilot and Chatgpt influence solo and pair programming. In *Proceedings of the 58th Hawaii International Conference on System Sciences*.
- [81] Hari Subramonyam, Divy Thakkar, Andrew Ku, Juergen Dieber, and Anoop K Sinha. 2025. Prototyping with prompts: Emerging approaches and challenges in generative AI design for collaborative software teams. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–22.
- [82] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2025. Human-in-the-loop software development agents. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 342–352.
- [83] Valerio Terragni, Annie Vella, Partha Roop, and Kelly Blincoe. 2025. The future of AI-driven software engineering. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–20.
- [84] Patanamon Thongtanunam, Shane McIntosh, Ahmed E Hassan, and Hajimu Iida. 2016. Revisiting code ownership and its relationship with software quality in the scope of modern code review. In *Proceedings of the 38th international conference on software engineering*. 1039–1050.
- [85] Jason Tsay, Laura Dabbish, and James Herbsleb. 2014. Let's talk about it: evaluating contributions through discussion in GitHub. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 144–154.
- [86] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards automating code review activities. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 163–174.
- [87] Asif Kamal Turzo and Amiangshu Bosu. 2024. What makes a code review useful to opendev developers? an empirical investigation. *Empirical Software Engineering* 29, 1 (2024), 6.
- [88] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [89] Liang Wang, Zhiwen Zheng, Xiangchen Wu, Baihui Sang, Jierui Zhang, and Xianping Tao. 2023. Fork entropy: Assessing the diversity of open source software projects' forks. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 204–216.
- [90] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *arXiv preprint arXiv:2509.14745* (2025).
- [91] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
- [92] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F Bissyandé, Yang Liu, and Haoye Tian. 2025. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. *arXiv preprint arXiv:2506.23749* (2025).
- [93] John Yang, Carlos E Jimenez, Alexander Wetteg, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [94] Ehsan Zabardast, J. Gonzalez-Huerta, Francis Palma, and Panagiota Chatzipetrou. 2023. Exploring the Relationship Between Ownership and Contribution Alignment and Code Technical Debt. <https://api.semanticscholar.org/CorpusId:257952678>
- [95] Fiorella Zampetti, Gabriele Bavota, Gerardo Canfora, and Massimiliano Di Penta. 2019. A study on the interplay between pull request review and continuous integration builds. In *2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 38–48.
- [96] Qunhong Zeng, Yuxia Zhang, Zhiqing Qiu, and Hui Liu. 2024. A First Look at Conventional Commits Classification. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 127–139.
- [97] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
- [98] Shurui Zhou, Bogdan Vasilescu, and Christian Kästner. 2019. What the fork: a study of inefficient and efficient forking practices in social coding. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 350–361.
- [99] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2024. Measuring github copilot's impact on productivity. *Commun. ACM* 67, 3 (2024), 54–63.